

Teleoperation

Easy wireless control for your homemade robot!



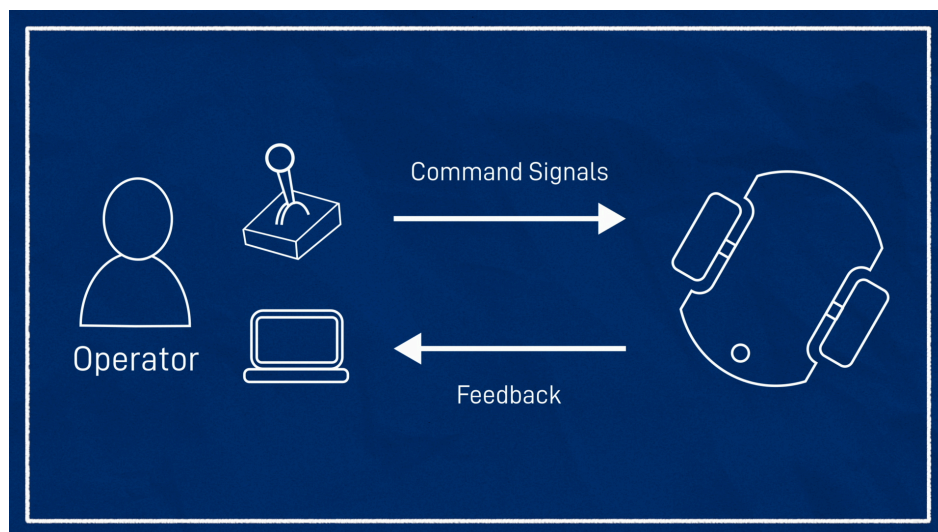
What is teleop?

Just like television and telephone mean distance vision and distance voice, teleoperation refers to controlling a robot from a distance, remote control. This is in contrast to autonomous operation, where the robot is given a task which it goes off and completes itself.

Although we like robots to be as autonomous as possible, in many instances human control is still required, especially during development or configuration.

Teleoperation usually consist of two parts:

- A way to send command signals (typically velocities) TO the robot
- A way to receive sensor data FROM the robot (this may be optional if you are physically present with the robot)



We have already seen an example of teleoperation in action. In the previous tutorials we used `teleop_twist_keyboard` to send `Twist` messages to our robot and we used `rviz` to view the camera and lidar feed it sent back.

It doesn't take long to realise though, that it's not a very practical approach. The keyboard:

- Only gives coarse control
- Requires terminal window to be kept open and active
- Isn't very "portable"

All of this can sometimes make for a less-than-comfortable teleop experience, so we're going to explore some other options we have to improve things!

Using a Gamepad

What is a gamepad?

You're probably familiar with gamepads - also known as game controllers. They're usually used for playing video games and are a compact and ergonomic way of controlling many buttons and axes. This tutorial will (try to) avoid using the term "controller" to minimise confusion with the controllers used by

ros2_control, but it will sometimes use the word "joystick", which is used by ROS and Linux to refer to all game controllers, not just actual joysticks (which are one large stick, gripped with a hand).

Gamepads generally fall into three main categories:

- Wired (USB)
- Wireless (USB)
- Bluetooth

Each has their pros and cons. Of course within each of these is a whole range of options with different features at different price points.

I personally prefer USB controllers (wired or wireless) to avoid pairing issues that sometimes occur with Bluetooth, especially when pairing to a robot with no GUI available for easy reconfiguration. Also, I prefer the ergonomics of the Xbox controller rather than PlayStation style, but everyone has their own preference.



The gamepad I used in the video is a cheap, wireless USB one (pictured centre above). If you want to get it, be warned that it is not a great gamepad: the thumbsticks have poor sensitivity, it constantly goes to sleep and needs to be woken up, changing the batteries is a pain... On the plus side though, it is cheap, wireless, and works out of the box with ROS, so it's worth picking up if you don't have an alternative.

Why use gamepads for robotics?

Gamepads are great for robotics because they're already designed to easily direct something to move around, it's usually just a virtual something. They typically have a few different axes and a whole lot of buttons, enough to trigger any functions you'd require.

With ROS, we can also decide whether we want to connect the gamepad directly to the computer in the robot, or to our base station/development machine. Connecting to the dev machine makes it easier when swapping from simulation to the real robot because you don't change anything, but you'll get better performance (with lower latency) connecting to the robot. There's nothing stopping you from setting up both, but this tutorial assumes everything is connected to the dev machine the whole time. If you want to do it the other way just remember to edit the files and run the commands on the Pi instead.

Gamepads in Linux

Most ROS users will be running Linux, which generally has ok support for gamepads (though not quite as good as Windows, since Linux doesn't have a history of being game-focused, something Valve is making huge leaps in changing).

So our first step is going to be checking that the gamepad actually works in Linux. Without going too deep into it, there are two different underlying interfaces that Linux can use to handle gamepads, the "old way" and the "new way". For the most part this isn't something we need to know about, and all the examples in this tutorial will use the "new way" underneath, but in the next tutorial there will be some extra info about the old way, in case your situation requires it.

To check our gamepad works in Linux, we want to install some useful tools:

```
sudo apt install joystick jstest-gtk evtest
```

With the controller connected (e.g. via USB or Bluetooth), we can open a terminal and run `evtest`. It will ask for the event number (it helpfully tells you some options), and after proceeding, any button presses should cause some new text to come through on the screen.

```
dev@dev-PC:~/dev_ws$ evtest
No device specified, trying to scan all of /dev/input/event*
Not running as root, no devices may be available.
Available devices:
/dev/input/event19: Controller
Select the device event number [0-19]: 19

Event: time 1666391436.219613, type 4 (EV_MSC), code 4 (MSC_SCAN), value 9000a
Event: time 1666391436.219613, type 1 (EV_KEY), code 313 (BTN_TR2), value 1
Event: time 1666391436.219613, type 3 (EV_ABS), code 9 (ABS_GAS), value 42
Event: time 1666391436.219613, ----- SYN_REPORT -----
Event: time 1666391436.227616, type 4 (EV_MSC), code 4 (MSC_SCAN), value 90001
Event: time 1666391436.227616, type 1 (EV_KEY), code 304 (BTN_SOUTH), value 0
Event: time 1666391436.227616, type 3 (EV_ABS), code 9 (ABS_GAS), value 153
Event: time 1666391436.227616, ----- SYN_REPORT -----
```

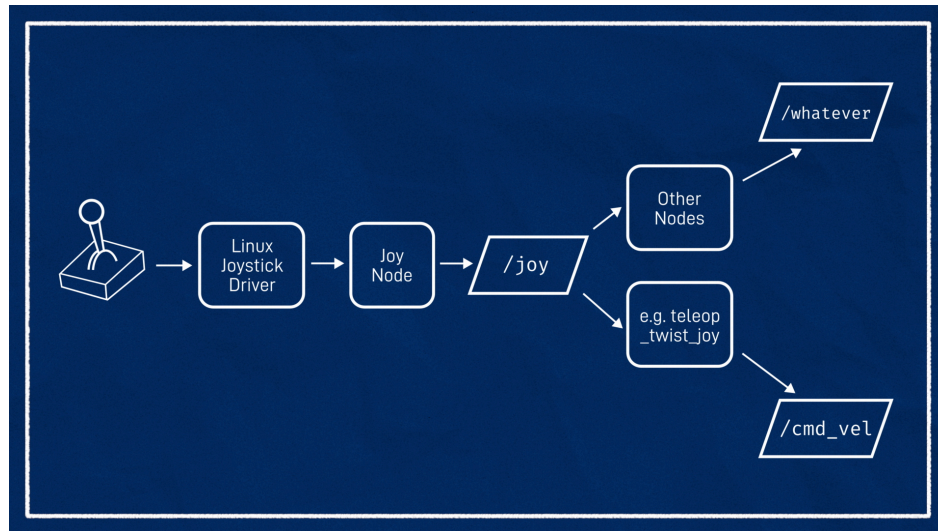
You can also experiment with `jstest` and its graphical counterpart `jstest-gtk` (which is pretty nice), but just be aware that they are using the old driver underneath.

Connecting to Joysticks/Gamepads in ROS

Once Linux can see the gamepad, we need to get it into ROS.

ROS comes with some packages that make working with gamepads and joysticks really easy. Rather than just running one program that generates `Twist` messages directly (like `teleop_twist_keyboard`), the process is split into two. The first program communicates with the joystick through the Linux drivers and publishes a `sensor_msgs/Joy` message - essentially a list of which buttons and axes were pressed at that time. This data can be used by other nodes to drive behaviour (e.g. trigger a service when a particular button is pressed, or use an analog stick to steer a camera with a servo). What we'll be using it for soon is to pick out the thumbstick values and publish an appropriate `Twist` message.

Although splitting things up might seem more complex, it means that we don't need to rewrite the `Twist` calculations for every joystick out there, and we don't need to rewrite the joystick driver code for every possible application.



To listen to a joystick with the new drivers, we can use the tools from the `joy` package. The first one will tell us which controllers ROS can see:

```
ros2 run joy joy_enumerate_devices
```

```
dev@dev-PC:~/dev_ws$ ros2 run joy joy_enumerate_devices
Joystick Device ID : Joystick Device Name
-----
0 : Controller
```

We usually want to use controller 0 (note that this ID is different to the one in `evtest`) which is the default, so for now there no need to specify parameters when we run the joy node (we will look briefly at some of the parameters in the next tutorial).

Note, I tried testing with two gamepads plugged in and actually had trouble specifying the second one and could only get it to function correctly with device 0.

Now we can open up two terminals to run the two commands below - the first is running the node and the second is displaying the outputs.

```
ros2 run joy joy_node # <-- Run in first terminal
ros2 topic echo /joy # <-- Run in second terminal
```

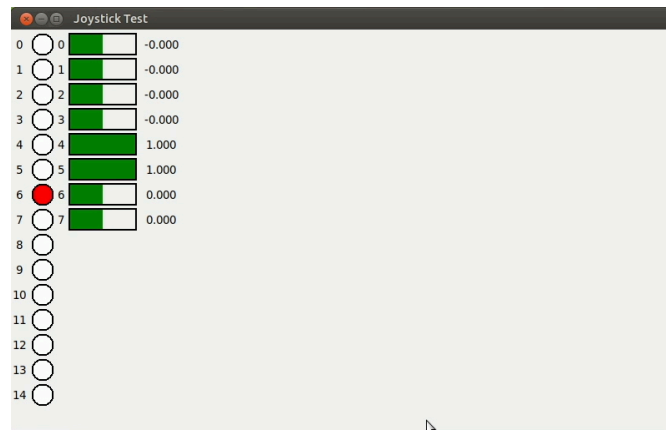
```
dev@dev-PC:~/dev_ws$ ros2 run joy joy_node
[INFO] [1666391605.782443697] [joy_node]: No haptic (rumble) available, skipping
initialization
[INFO] [1666391605.782692711] [joy_node]: Opened joystick: Controller. deadzone
: 0.050000
```

If everything works correctly, we should see messages coming through on `/joy` as we press the buttons and move the sticks.

Bonus: A nicer way to check joysticks

Echoing `/joy` is good for checking that the gamepad is working, but it isn't always easy to tell exactly what's going on, especially counting which axis is which. I couldn't find any other obvious tools out there for this so I wrote a simple one myself, available [here](#). It's not currently available through the package manager so you'll have to install it from source (but I may release it if there is interest).

Once installed, it can be run with `ros2 run joy_tester test_joy`.



This is a good opportunity to note down which axis we want to use for driving forward and backwards, which one is for steering left and right, and also the two buttons we want to use for selecting which speed mode to use - more on that soon.

Creating a launch and params file

Before we go any further, we're going to create a launch file and a params file for running our gamepad control. That's because the next node we'll be running has a lot of parameters to specify, and typing them all in at the command line will be tedious.

We'll start with the launch file. Create `joystick.launch.py` in the `launch` directory, and copy-paste the launch script below. This script simply:

- Specifies the path to a params file
- Declares a joy node that uses the params file
- Launches it

```
from launch import LaunchDescription
from launch_ros.actions import Node

import os
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():

    joy_params = os.path.join(get_package_share_directory('articubot_one'),'config','joystick.yaml')

    joy_node = Node(
        package='joy',
        executable='joy_node',
        parameters=[joy_params],
    )

    return LaunchDescription([
        joy_node
    ])
```

Of course we then need to actually write the params file, so we will go ahead and create `joystick.yaml` in our config directory. The parameters specified below are only a subset of the [full parameter list](#) but they are ones we are more likely to change. The values shown here are actually the same as the defaults, but they are there to change if we need to. We'll learn a bit more about them in the next tutorial.

```
joy_node:
  ros_parameters:
    device_id: 0
    deadzone: 0.05
    autorepeat_rate: 20.0
```

After a rebuild to add the new files, we should be able to launch `joystick.launch.py` and get the same behaviour as we did running `joy_node` directly.

Converting Joy to Twist

Now that we have our raw joystick data coming through on `/joy` we need to convert it to a `Twist` message. The package we use for this is called `teleop_twist_joy`.

Although we *can* run it at the command line with `ros2 run`, there are so many nested parameters to set that it's not worth it unless we're trying to troubleshoot something. Instead, let's add it straight to our launch file and we'll pass the parameters using the same `joystick.yaml` file as earlier to keep things simple.

By default, `teleop_twist_joy` wants to publish its output to `/cmd_vel`, but remember our differential drive controller from the last tutorial is subscribing instead to `/diff_cont/cmd_vel_unstamped`, so we need to remap the topic name. **Also note** the example code below explicitly names the node `teleop_node`. This is because the default "node name" (that we need to set the parameters properly) doesn't match the "executable name" which, while technically fine, is quite confusing. By explicitly naming it, we force everything to line up.

```
teleop_node = Node(
    package='teleop_twist_joy',
    executable='teleop_node',
    name = 'teleop_node',
    parameters=[joy_params],
    remappings=[('/cmd_vel', '/diff_cont/cmd_vel_unstamped')])
```



```

    )
#...

# And add to Launch description at the bottom

```

Now for the parameters. For each of the six dimensions of the `Twist` we can specify:

- Which joystick axis to use
- Max speed in regular mode
- Max speed in turbo mode

Keep in mind that our differential drive robot controller can only linear X and angular Z, so we only need to set those.

We also need to set which button enables regular mode, and which button for turbo mode. Having separate regular and turbo speeds gives us finer control over the robot, especially if your joystick sensitivity is poor or you're just using the D-pad.

Finally, we have the option of disabling the "dead-man's switch". By default, you need to keep the regular or turbo button held down while moving the joystick to send commands, so that if something bad happens (e.g you drop the controller and the thumbstick gets bumped), it won't send any motion commands. This is enabled by default, and the parameters below keep it that way, but you can change it if desired.

So we've got `axis_linear`, `scale_linear`, `scale_linear_turbo`, then the same for `*_angular_*`. Inside all the linear ones we want to set what it is for the x dimension, and in the angular ones we have `yaw` (note that it is `roll`, `pitch`, `yaw` here even though the resulting `Twist` message will refer to the angular axes also as `x`, `y`, `z`). Finally we've got the `require_enable_button`, `enable_button`, and `enable_turbo_button` button.

The example below sets the regular speeds to 0.5 (m/s and rad/s) and the turbos to 1. These are decent defaults and you can tweak them now or later if required. If you can't remember your axes or button numbers that's ok, just have a guess so that we can get this to run, and then you can rerun the test tool to confirm it later.

```

# ... below the joy parameters

teleop_node:
  ros__parameters:

    axis_linear: # Left thumb stick vertical
      x: 1
    scale_linear:
      x: 0.5
    scale_linear_turbo:
      x: 1.0

    axis_angular: # Left thumb stick horizontal
      yaw: 0
    scale_angular:
      yaw: 0.5
    scale_angular_turbo:
      yaw: 1.0

    require_enable_button: true
    enable_button: 6 # Left shoulder button
    enable_turbo_button: 7 # Right shoulder button

```

Once those are in we can rerun the launch file. In a new terminal, run `ros2 topic echo /cmd_vel` to see the values coming up.

When we move the sticks by themselves there should be nothing, when we hold the regular speed button down, we should get those regular speeds, and likewise the higher speeds with the turbo button.

Driving the robot

Now that we are generating command velocities, let's check that they work with the robot. The safest way to do this is to test first in Gazebo, then with the real robot propped up, then with the robot on the ground.



Getting feedback

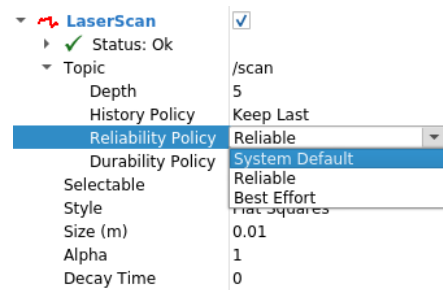
The other half of teleoperation is feedback - making the robot's sensor data available to the operator. When we're using ROS, our first port of call for feedback is of course going to be RViz - we can use it to display just about anything we need. That's not to say it's always the best option - some

applications will require custom tools, and there are other general purpose viewers available like Foxglove (which we'll explore in the next tutorial), which are quite good.

The examples below are assumed to be in RViz.

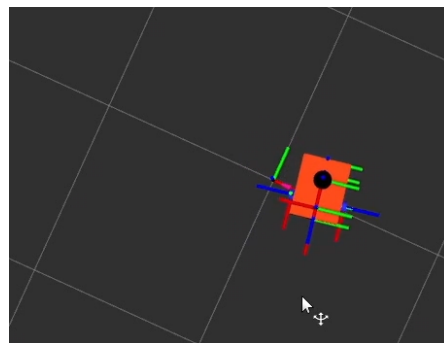
QoS

Before looking at these examples, it's worth a quick note about QoS. Quality-of-Service (QoS) is a feature of the ROS messaging system that allows nodes to control some technical aspects of how they send and receive data. Depending on the settings of both the publisher and the subscriber, messages may or may not be getting through. If you are having trouble getting something to show in RViz, the first thing to check is the QoS settings, and particularly try setting the reliability to *Best Effort*. If QoS is restricting you, there should be a warning printed in the RViz terminal window.



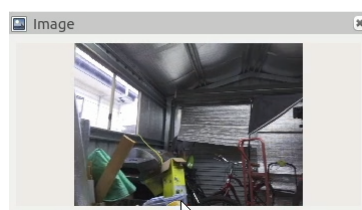
Odometry

The first feedback we get for free is our odometry (from the motor encoders, via `ros2_control`). By watching the robot model (or the axis markers) in the 3D view we get a rough sense of where our robot is moving. However, we have no understanding of the environment and odometry is prone to drift over time, so it's not that useful on its own.



Camera feed

One of the more natural kinds of feedback is of course the camera, as this mimics what we see with our own eyes. We can open an *Image* display in RViz (or a *Camera* display).



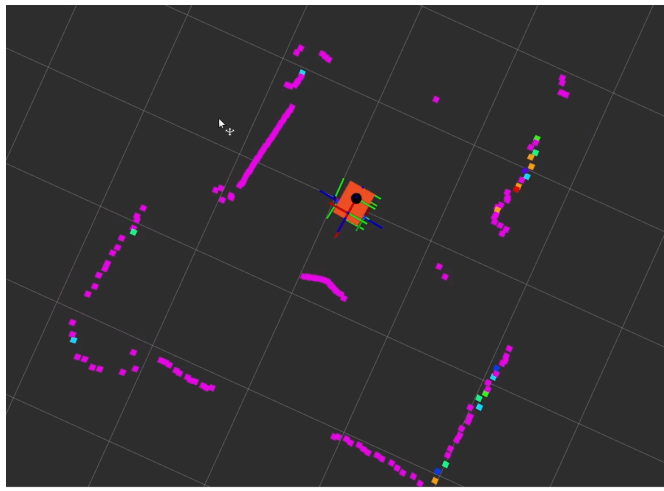
Unfortunately, something you may notice is that the image feed in RViz is choppy and delayed. This is because (*I think*) in Foxy, RViz doesn't use `image_transport` to subscribe to images, which means it doesn't know how to use compression, and so the image feed takes up too much network bandwidth from the Pi. This should be fixed in newer versions of ROS, but in the meantime we can just look at images in other ways, e.g. with `rqt_image_view`.

You may also find that reducing the framerate in the camera driver helps. For the `v4l2_camera` node used in these tutorials, this can now be done with the `time_per_frame` parameter which takes an array of two integers to create a fractional interval (e.g. `[1, 30]` = $1/30\text{s}$ = 30Hz).

Note, I have found huge inconsistencies with the ability to stream images, especially in areas of busy WiFi. Some times, in some places, it works great, other times many frames are dropped.

Lidar feed

The other sensor we have on our robot is lidar. By monitoring the lidar feed in RViz we can get a pretty good sense of the robot's location in indoor spaces, especially corridors.



Other feedback and alternative viewers

Depending on your sensors and algorithms there may be other data that is desirable to display. For example, if you are running a SLAM algorithm that is generating a map, visualising that would be helpful.

More creatively, we could set up feedback through other senses such as sound (e.g. beeping when close to an obstacle or even passing data from a microphone), or touch (e.g. haptic feedback, or "rumble", on a controller).

There are also other tools available for displaying our data, some of which we'll explore in the next post.

Conclusion

Now we are able to drive our robot around with a gamepad and use RViz to receive feedback. Take some time to have fun driving it around!

**Markus_Messerschmidt**

Dec '22

Hello.
 which ip addresses are taken? first line that of the robot and in the second line that of the dev machine?
 Unfortunately, the npm server is not running as desired for me.
 Do I have to rebuild the root directory if there are changes or just run npm install and then npm start in the example folder?

[1 reply](#)**JoshNewans**

Jan '23

Markus_Messerschmidt:

which ip addresses are taken? first line that of the robot and in the second line that of the dev machine?

Are you referring to [here](#) where I set the IP addresses in the web page?

The IP address for the ImageStream should be whichever is running web_video_server and the one for RealGamepad/SimGamepad should be whichever is running rosbriidge. In my example the former was the Pi and the latter was the dev machine. Make sure you do it for both of the pages (sim and real).

Markus_Messerschmidt:

Unfortunately, the npm server is not running as desired for me.

In what way?

Markus_Messerschmidt:

Do I have to rebuild the root directory if there are changes or just run npm install and then npm start in the example folder?

Yeah, good question, I get very confused by it all. In theory when you npm start it should auto pick up changes. I think in this case because there are actually two levels, the library and the example page, it doesn't seem to work properly. And to make things confusing the "conceptual top level" (the example page) code is nested *inside* the lower level (the library).

So if you are only tweaking something at the example level you should not need to rebuild, but if you tweak the library code I think you need to rebuild it then rerun the example/page/server bit.

This is an area I am *very* weak in so I probably did it all wrong, and I also wrote it a couple of years ago.

I am (slowly) working on a Foxglove extension to totally replace this which will be a far better approach!

**Sasffm**

Feb '23

Hi there,
 i am having a strange behaviour.
 While using a gamepad my robot turns right when i am steering to the left and it turns left when i am steering to the right.
 Where is my mistake here?
 Thanks

[1 reply](#)**JoshNewans**[▶ Sasffm](#) Feb '23

Hey!
 The simplest answer is probably: in the teleop_twist_joy params, try just flipping the sign of the angular Z value.
 You can also step through and echo /joy and your cmd_vel topic to check what the values are at each stage.

[2 replies](#)**Sasffm**[▶ JoshNewans](#) Feb '23

Thanks for the fast answer.
 Meanwhile i have found the problem.
 I had to switch the motor cables on the controller.

**John_Nam**

Mar '23

Thank you Josh your great video.
 my robot turn left with joystick but my rviz turn right...
 in your tutorial... you explained I think.
 but I couldn't find your explain...
 where should I fix it...
 Anyone knows this issue and how to fix it please let me know

Regards,

John...

[1 reply](#)**John_Nam**

Mar '23

JoshNewans:

an also step through and echo /

fixed with hardware connection ... motor connection with the arduino.

**John_Nam**

Mar '23

interrupt each other → camera.launch.py and launch_robot.launch.py
 in ros2_control.xacro

/dev/serial/by-path/platform-fd ~~~~ tried avoid interrupt arduino and rplidar
 rplidar.launch.py and launch_robot.launch.py is ok

issue: during using rviz2 -d ~~~... can not use camera...

[1 reply](#)**JoshNewans**[▶ John_Nam](#) Mar '23

Hey John, sorry I'm a little confused by your replies - can you please clarify what the current problems are?

- The first one seemed to be an issue with it showing the turns the wrong direction in RViz, but then possibly you got that fixed?
- The second I'm not sure if you are saying there is a clash between the camera and the lidar? Or the Arduino?

If the camera is not showing up in RViz, make sure you are using an Image display (not a Camera as it can be trickier to get going) and check with the dropdown that the topic is actually being published. There was an issue I didn't explain clearly at this point that the simulated camera and real camera were set up to publish on slightly different topics.

If you can explain the issue more clearly I'm happy to try and help 😊

1 reply



John_Nam

JoshNewans Mar '23

There is a clash between the camera and the Arduino.
the lidar and the Arduino(robot) looks fine at the moment.

in the ros2_control.xacro

```
diffdrive_arduino/DiffDriveArduino
left_wheel_joint
right_wheel_joint
30
```

```
<param name="device">/dev/ttyUSB1</param>
```

```
**usb0 is lidar, usb1 is robot
---of----
```

```
/dev/serial/by-path/platform-fd500000.pcie-pci-0000:01:00.0-usb-0:1.3.1:1.0-port0
```

both I tested but same issue...

```
<param name="baud_rate">57600</param>
<param name="timeout">1000</param>
<param name="enc_counts_per_rev">3436</param>
</hardware>
```

in the pi

```
mate_bot@pi:~$ ls /dev/ttyUSB*
/dev/ttyUSB0 /dev/ttyUSB1
```

```
mate_bot@pi:~$ ls /dev/serial/by-path
platform-fd500000.pcie-pci-0000:01:00.0-usb-0:1.3.1:1.0-port0
platform-fd500000.pcie-pci-0000:01:00.0-usb-0:1.3.3:1.0-port0
```

in the rviz2

reset button

robot and lidar working

camera warn... camera info: No camera info received on [/camera/camera_info] Topic may not exist

if only run camera launch file, rqt_image_view is working.

Please let me know how to fix the crash with camera and robot(arduino)
I searched google it but couldn't find the solution
sorry interrupt and my poor English

1 reply



John_Nam

Mar '23

camera issue with web server.
any one have an idea please let me know

<http://localhost:3000/realgamepaddemo>

There is no video stream.

I check the topic list at the dev machine.

There is a /image_raw as below.

```
dev-pc@Dev-PC:~/dev_ws$ ros2 topic list
/camera_info
/diff_cont/cmd_vel_unstamped
/image_raw
/image_raw/compressed
/image_raw/compressedDepth
/image_raw/theora
/joy
/joy/set_feedback
/parameter_events
/rosout
```

Tested with the rqt_image_view is working at the dev machine
dev-pc@Dev-PC:~/dev_ws\$ ros2 run rqt_image_view rqt_image_view

The pi side web_video_server is working well I think
mate_bot@pi:~/robot_ws\$ ros2 run web_video_server web_video_server
[INFO] [1679537318.652438037] [web_video_server]: Waiting For connections on 0.0.0.0:8080

1 reply



JoshNewans

Mar '23

Hey John, ok let's unpack a few of these things.

1 John_Nam:

There is a clash between the camera and the Arduino.

Unless you have a camera that works over serial, it shouldn't be possible for these to clash.

1 John_Nam:

in the rviz2

reset button

robot and lidar working

camera warn... camera info: No camera info received on [/camera/camera_info] Topic may not exist

if only run camera launch file, rqt_image_view is working.

Ah I think this might be a problem that I've been meaning to look into. In RViz there are two ways to look at camera data - an "Image" display and a "Camera" display. The Camera display expects there to be not just an image topic, but a CameraInfo topic to go along with it. When we simulate with Gazebo it creates both topics, but I think the camera driver I'm using is NOT creating the CameraInfo topic. I think this might be because it doesn't have calibration data but I'm not 100% sure.

Everything should work fine regardless, as all the other tutorials rely only on the image data not the camera info. And in RViz you can just change it to an "Image" display.

1 John_Nam:

(post deleted by author)

I saw you've deleted this one, but to answer the question anyway in case someone else finds this: if you are running it with the "real controller" it won't show up the buttons until it has seen the gamepad. Usually that will happen the first time you press a button after loading the page.

1 John_Nam:

camera issue with web server.

any one have an idea please let me know

<http://localhost:3000/realgamepaddemo>
There is no video stream.
I check the topic list at the dev machine.

At a guess, I'd say you're not running the web video server which needs to also be running and it converts the image topics to a web-friendly format. [Here](#) is the repo for it, I do show that in the "Part B" video.

As a final note, I am still working on my plugin for Foxglove that makes that whole janky web demo completely redundant, so stay tuned for that 🤖

Hope that helps, please reply if you have further troubles.

1 reply



John_Nam

Mar '23

for the rviz2
git clone issue...

git clone -b port_plugin-tutorials [GitHub - ros-visualization/visualization_tutorials: Tutorials related to using and extending RViz and interactive markers.](#)

cloning into 'visualization_tutorials'...
fatal: Remote branch port-plugin-tutorials not found in upstream origin

Thank you very much your wonderful lecture
I am not an IT person... so sometime very easy things to you... very difficult to me...
your 20 min lecturing... to me more than 1 week learning...
Why so many stupid questions and issues



MarcW

▶ JoshNewans Mar '23

Hey Josh,
I use a GStreamer node that I found on github: [GitHub - clydemcqueen/gscam2: ROS2 camera driver for GStreamer-based video streams, supports intra-process communication](#)
It's a ROS2 port, publishes the camera info and worked fine for me. The Jetson Nano has some issues with MIPI cameras which is why I went this way.

1 reply



RobotRoss

▶ MarcW Apr '23

Teleop 14 Part A. Thank you for this excellent tutorial which brings together essential info on understanding teleoperation. And the joy_tester package is very helpful in determining the Button and Axis numbering in a ROS2 system-not clearly explained anywhere else as clearly, and pleased that it now can do a binary instal on a Ubuntu 22.04 machine, ros-humble-joy-tester.



Rakshit_Soni

May '23

Hey Josh,
I am trying to subscribe the Joy message from joy_linux node to a custom node. Basically I want to convert the Joy message to Float64 for speed(linear) and float64(wheel angle) for a robot behaving like Ackermann steered. I have written the pub/sub nodes for these two messages but getting error in the code. Also, for Ackermann steered robot, how I can control the robot. As there is no support for Ackermann steered robot in ROS2, no ros2-control is there, no gazebo plugin except for toyota prius example but that doesn't work. I am new to ROS so need some advice in these issues.
If you can help me with this, it will be great. Using ROS2 Humble, Ubuntu 22.04

THE CODE:

```
#!/usr/bin/env python
import sys
import rclpy
import pdb
from sensor_msgs.msg import Joy
from std_msgs.msg import Float64
import subprocess, time
from rclpy.node import Node
from std_msgs.msg import String

def rescale(x, oldmin, oldmax, newmin, newmax):
    r = (x-oldmin)/(oldmax-oldmin)
    out = newmin + r*(newmax-newmin)
    return out

class Joystick2speed(Node):

    def __init__(self):
        super().__init__('joystick2speed')
        self.pub = self.create_publisher(Float64, 'speedcmd_metersec', 1)
        self.sub = self.create_subscription('/joy', Joy, self.joy_callback, 1)
        self.buffer = []
        for i in range(0,100):
            self.buffer.append('X')
        self.buffer_index = 0
        self.get_logger().info("Speed node has been initialized")
    def joy_callback(self, data):
        joystick_y = -data.y
        deadmanzone_min = -0.2
        deadmanzone_max = 0.2
        max_fwd_speed_per_second = 2
        max_bkwd_speed_per_sec = 2
        if joystick_y > deadmanzone_min and joystick_y < deadmanzone_max:
            velocity = 0
        elif joystick_y > 0:
            velocity = rescale(joystick_y, deadmanzone_max, 1., 0., max_fwd_speed_per_second)
        elif joystick_y < 0:
            velocity = rescale(joystick_y, -1., deadmanzone_min, -max_bkwd_speed_per_sec, 0.)
        msg_out = Float64(velocity)
        print(velocity)
        self.pub.publish(msg_out)

def main(args = None):
    rclpy.init(args=args)
    joystick2speed = Joystick2speed()
    rclpy.spin(joystick2speed)
    rclpy.shutdown()
    if __name__ == '__main__':
        main()
```

Error: Traceback (most recent call last):
File "/home/computing/podcar_2/src/pod2_bringup/scripts/joystick2wheelAngle.py", line 34, in main()
File "/home/computing/podcar_2/src/pod2_bringup/scripts/joystick2wheelAngle.py", line 30, in main
joystick2wheel = Joystick_2_wheel()
File "/home/computing/podcar_2/src/pod2_bringup/scripts/joystick2wheelAngle.py", line 16, in init
self.pub = self.create_publisher('wheelAngleCmd', Float64, 10)

```
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rcipy/node.py", line 1273, in create_publisher
final_topic = self.resolve_topic_name(topic)
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rcipy/node.py", line 1226, in resolve_topic_name
return _rcipy.rcipy_resolve_name(self.handle, topic, only_expand, False)
TypeError: rcipy_resolve_name(): incompatible function arguments. The following argument types are supported:
1. (arg0: rcipy::Node, arg1: str, arg2: bool, arg3: bool) -> str

Invoked with: <rcipy._rcipy_pybind11.Node object at 0x7ffb31f413b0>, <class 'std_msgs.msg._float64.Float64'>, False, False
```

S

Stardobas

Nov '23

Hey Josh,

I've destroyed all system trying to make "control your robot from your phone"... 😞 I'm not able to install nothing (apt install) anymore, this give me a strange irreversible error in dependencies...(navigation need cereslib that can't be installed for example...).

Now, first I try again in new system, need to know if in your method (node.js...stream camera...game pad controller) **is possible to stream Rviz too...?**... I would be have camera, controller, and Rviz working...

PS, in my system I don't use internet but only local net, and now I'm trying to work all system in raspberry, using navigation and mapping too...is very slow but I think in future the pi5 help more to do this. Following your fantastic tutorial in my pc (simulated robot) all work except remote control from web because I'm afraid about trying tutorial to destroy pc system too like raspberry).

Thank you

F

Foolsim_simulate

Apr '24

currently, i can run ros2 run rosbridge_server rosbridge_websocket on workspace ros-ui-react/example is succesfull , so I need to install git clone [GitHub - RobotWebTools/rosbridge_suite: Server Implementations of the rosbridge v2 Protocol](#) on workspace : Dev_ws or not ? because i tried to use a joystick. Launch is a real robot but that does not move (I have adjusted joystick.launch.py)

R

RawStone

Aug '24

Hey Josh! Thank you so much for the lecture series.

I am trying to run the articubot_one repo on my own system and hardware. I've made all the proper motor connections and have gotten all the launch-scripts and nodes to work.

I've run into a weird error and I will try my best to describe it so bear with me.

When I try to operate the robot via Teleop commands, the robot only takes the command once every few seconds. It moves once every few seconds and stops after its 'set' timeout period(which according to my code is 2secs). i.e. if it doesn't receive the next commands with in 2 seconds of the prev command, the motor stops moving. Now i call this behaviour random because sometimes the motor receives the commands 2 seconds apart from each other, sometimes they're 5 seconds apart, sometimes even 30 secs apart. It does not move in those brief periods of time.

- I have tried running the robot via miniterm and I continuously kept pasting the command "o 255 255". The motor constantly keeps moving for however long i keep pasting the command which leads me to believe there is nothing wrong in the circuitry corresponding to Arduino.
- I have checked my keyboard and made sure that there is nothing wrong with the keys & keystroke signals are continually sent.

Can you suggest what else I can do or if anyone has any idea what could be wrong, help would be greatly appreciated.

Thanks in advance

1 reply

R

RawStone

RawStone

Oct '24

So I found the issue.

The problem appears to be with my motor of choice. well the encoder on the motor is whats causing the problem to be precise. Instead of using the provided 6pin female header, i bypassed it and gave the connections directly to the motors on their terminals on the bottom and worked like a charm. I'll have to figure out what exactly is causing this problem on the encoder.

Leaving this here in case anyone in the future has a similar issue.

Cheers!

[Continue Discussion](#)



